Introduction to Programming:

DECISION MAKING AND LOOPING

Prepared By:

Darsha Chauhan

Outline

- Introduction to Loops
- While Loop
- Do While Loop
- For Loop
- Break Statement
- Goto Statement
- Continue Statement
- Sample Programs.

Loop

- Loop is used to execute the block of code several times according to the condition given in the loop.
- It means it executes the same code multiple times.
- For example: O/p: Hello – 5 times

```
printf("Hello\n");
printf("Hello\n");
printf("Hello\n");
printf("Hello\n");
printf("Hello\n");
```

```
Loop(Condition)
{
        //statement
}
```

Loop

- Loop Control in C, is the repeated execution of code, for a specified number of times,
- if the condition is met.

```
loop(50 times)
{
        printf(*);
}
```

Looping or Iterative Statements in C

- Entry Controlled Loop: while, for
- Exit Controlled Loop: do…while
- Virtual Loop:goto

Loop: while

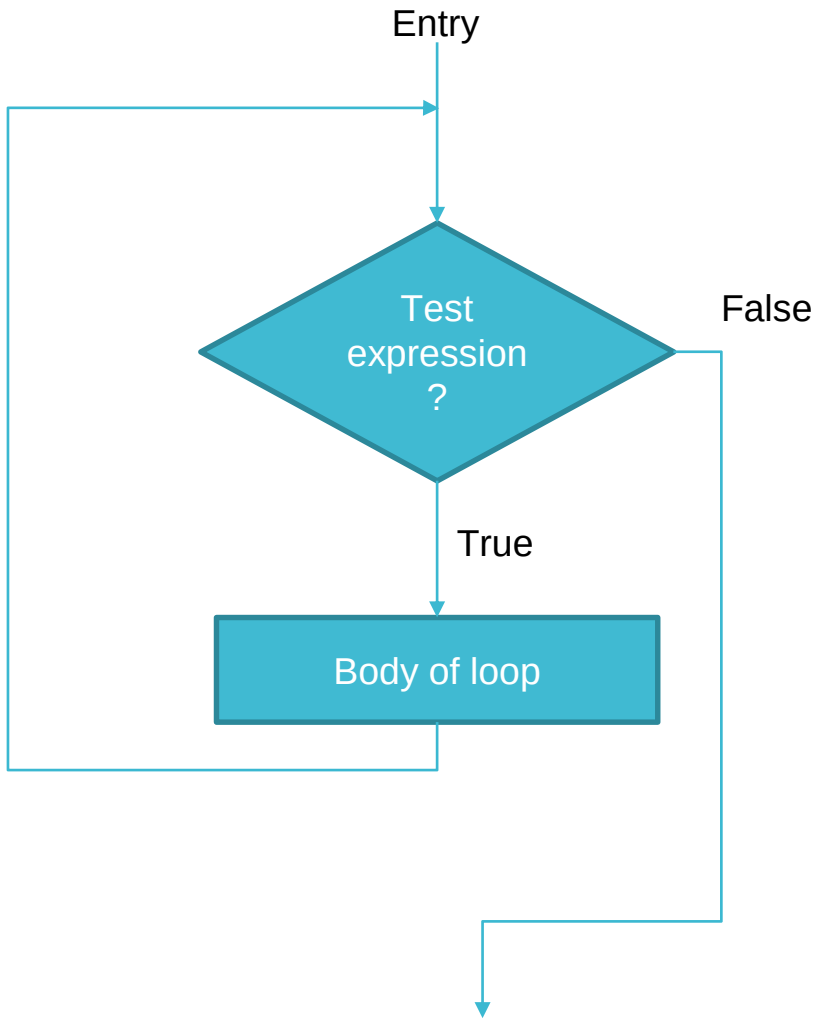
- while is an entry controlled loop
- Statements inside the body of while are repeatedly executed till the condition is true
- while is keyword
- Syntax:

```
while(test condition)
{
        // Body of the while
}
```

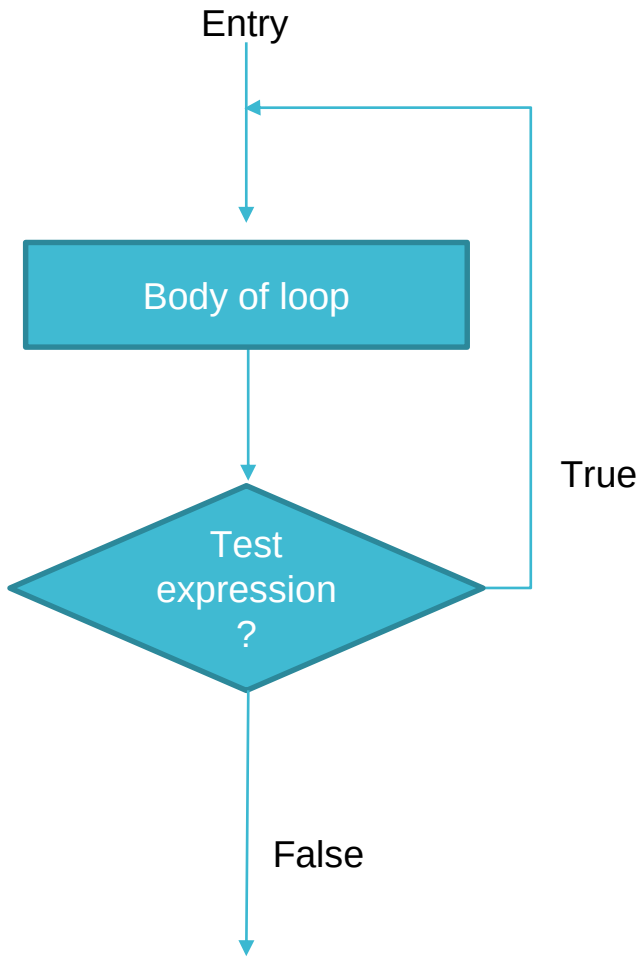


Loop statement

- Entry Controlled Loop



Exit Controlled Loop



Example:

Write a program to print 1 to n(while loop)

```
#include<stdio.h>
void main()

{

        int  i,n;
        i=1;

        printf("Enter Number:");
        scanf("%d",&n);

        while(i<=n)

        {

                printf("%d\n",i);

                i=i+1;
        }

}
```

O/p:

Example:

Write a program to print Odd numbers between 1 to n(while loop))

```
#include<stdio.h>
void main()

{

        int  i,n;
        i=1;

        printf("Enter Number:");
        scanf("%d",&n);

        while(i<=n)

        {
                if(i%2!=0)
                        printf("%d\n",i);

                i=i+1;
        }

}
```

O/p:

Example

- Write a program to print multiplication table(while loop)
- Write a program to Sum of 5 numbers entered by user(while loop)



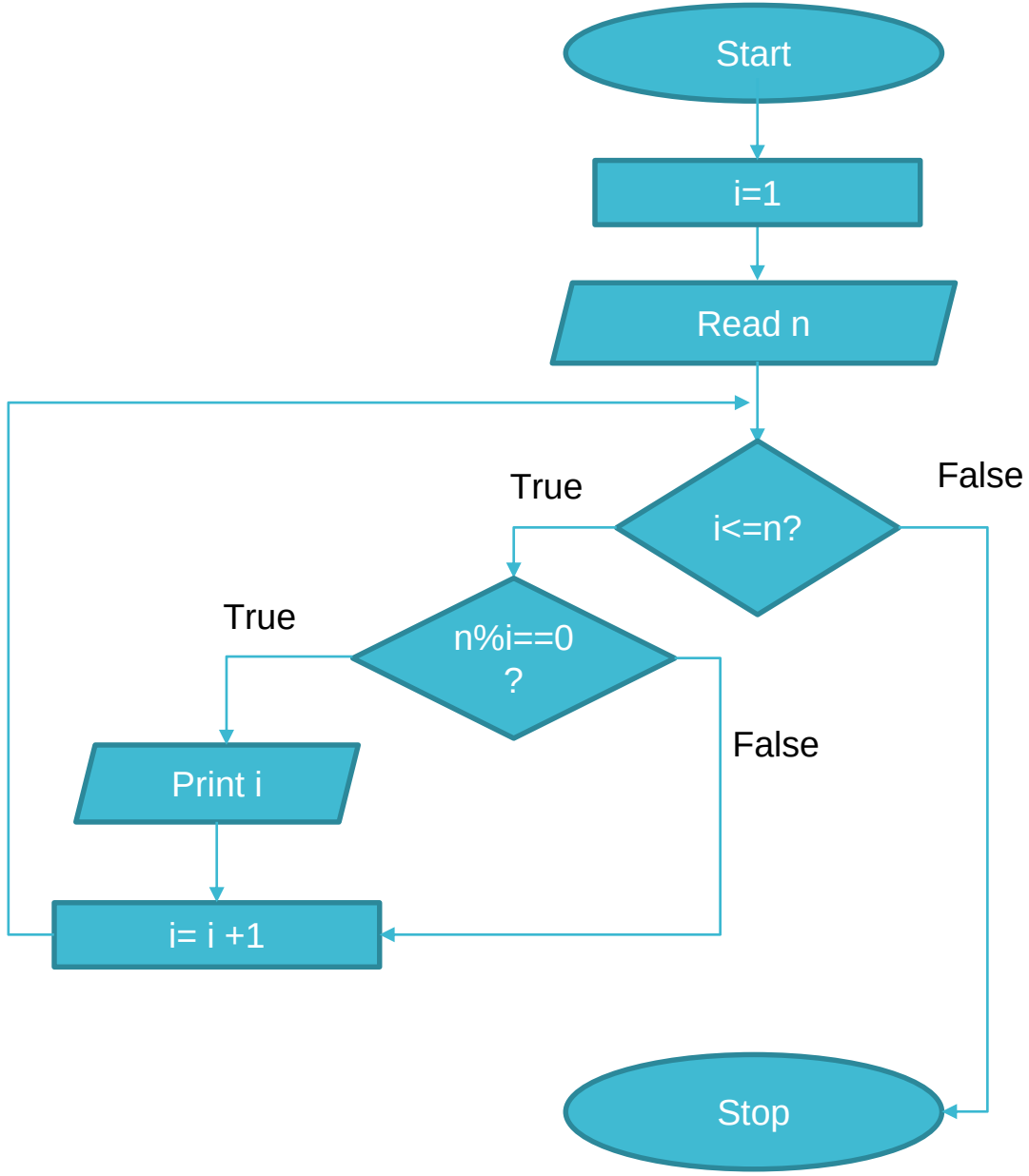
How to build logic?

- Step 1: Understand the problem statement
  - e.g. Write a program to find factors of a number.
  - Run following questions through mind
- What is the factor of a number?
  - Factor is a number that divides another number evenly with no remainder.
  - For example, 1,2,3,4,6,12 are factors of 12.
- How many variables needed? What should be their data types?(Inputs/Outputs)
  - To get number from user we need variable n.
  - Now we need to divide n with 1,2,3,…,n. For this we will declare a loop variable i initialized as 1.
  - Both variables should be of integer data type.
- What control structure you require?
  - First we need a loop to divide n by 1,2,3,…,n, loop will start from 1 and ends at n.
  - Inside loop we need if structure to check n%i==0 (Number n is evenly divisible by i or not).



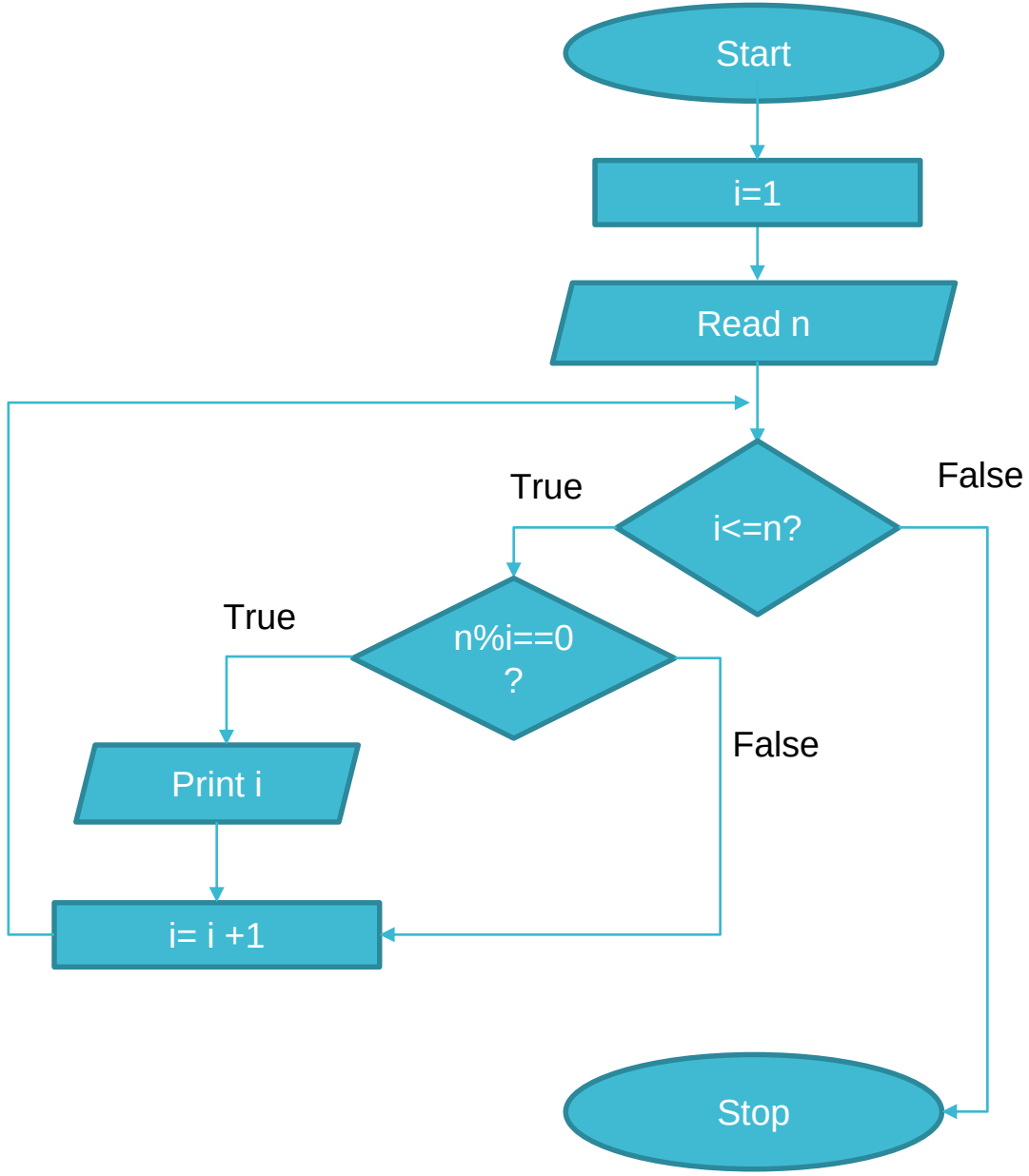
How to build logic?

- Step 3: Draw flowchart/steps on paper or in mind



How to build logic?

- Step 3: Draw flowchart/steps on paper or in mind



Step 1: Start

Step 2: Declare variables n,i

Step 3: Initialize variable

i ← 1

Step 4: Read value of n

Step 5: Repeat the steps until i <= n

5.1: if n%i == 0

Display i

5.2: i=i+1

Step 6: Stop

How to build logic?

- Step 4: Writing Pseudo-code
- Pseudo-code is an informal way to express the design of a computer program or an algorithm.
- It does not require any strict programming language syntax.

```
Initialize i=1 integer
Declare n as integer
Input n
while i<n
        if n%i
                print i
        end if
        increment i=i+1
end while
```

Example:

Write a program to find factors of a number

```
#include<stdio.h>
void main()
{
        int i=1,n;

        printf("Enter Number:");
        scanf("%d",&n);

        while(i<=n)
        {
                if(n%i==0)
                        printf(“%d”,i)
                i = i +1;;
        }

}
```

Loop: do…while

- do while is an exit controlled loop.
- Statements inside the body of do while are repeatedly executed till the condition is true.
- Do and while are keywords.
- Syntax:

```
do
{
    // statement
}
while (condition);
```

- Loop body will be executed first, and then condition is checked.
- If the condition is true, the body of the loop is executed again and the condition is evaluated.
- This process goes on until the condition becomes false.
- If the condition is false, the loop ends.

Example: do...while

Write a program to print Odd numbers between 1 to n

```
#include<stdio.h>
void main()

{

        int  i,n;
        i=1;

        printf("Enter Number:");
        scanf("%d",&n);

        do
        {
                if(i%2!=0)
                        printf("%d\n",i);

                i=i+1;
        }
        while(i<=n);

}
```

O/p:

Example: do...while

Write a program to find factors of a number

```
#include<stdio.h>
void main()
{
        int i=1,n;

        printf("Enter Number:");
        scanf("%d",&n);

        do
        {
                if(n%i==0)
                        printf(“%d”,i)
                i = i +1;
        }
        while(i<=n);

}
```

Example

- Write a program to print reverse a number
- Write a program to check given number is perfect or not
- Write a program to check given number is prime or not



Loop: for

- for is an entry controlled loop
- Statements inside the body of for are repeatedly executed till the condition is true
- for is keyword
- Syntax:

```
for (initialization; test-condition; increment)
{
        // statements / Body of loop
}
```

- The initialization statement is executed only once.
- Then, the condition is evaluated. If the condition is false, the for loop is terminated.
- If the condition is true, statements inside the body of for loop are executed, and the increment/decrement statement is updated.
- Again the condition is evaluated.

for Example

Write a program to print Numbers 1 to n

```
#include<stdio.h>
void main()
{
		int i,n;

		printf("Enter Number:");
		scanf("%d",&n);

		for(i=0 ; i<=n ; i++)
		{
			printf(“%d\n“,i);
		}
}
```

o/p:

```
Enter number: 3
1
2
3
```

for Example

Write a program to find factors of a number

```
#include<stdio.h>
void main()
{
        int i,n;

        printf("Enter Number:");
        scanf("%d",&n);

        for(i=0 ; i<=n ; i++)
        {
                if(n%i == 0)
                        printf(“%d,“,i);
        }
}
```

o/p:
Enter number: 12
1, 2, 3, 4, 6, 12

for Example

Write a program to check given number is perfect or not

```
#include<stdio.h>
void main()
{
        int i,n,sum=0;

        printf("Enter Number:");
        scanf("%d",&n);

        for(i=0 ; i<=n ; i++)
        {
                if(n%i == 0)
                {
                        printf(“%d+“,i);
                        sum=sum+i;
                }
        }
        printf(“=%d”,sum);
}
```

o/p:
Enter number: 12
1, 2, 3, 4, 6, 12

for Example

```
        if(sum==n)
                printf("\n%d is a perfect number",n);
        else
                printf("\n%d is not a perfect number",n);
}
```

o/p:

```
Enter number:6
1+2+3=6
6 is a perfect number
```

Nested for loop

- Nesting of loops = one for statement within another for statement
- For example, two for loops can be nested as follows:

```
-------
for(i=1; I<10; i++)
{
        -----
        for( j=1; j!=5; j++)
        {
                ---
        }
        ------
}
```

- The loops should be properly indented so as to enable reader to easily determine which statements are contained with in each for statement
- Generally: outerloop determines row &

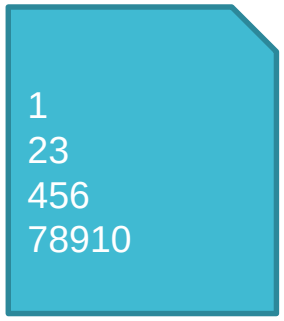
  innerloop determines columns

Nested for loop Pattern

- There are important points to note in pattern
- Determine, how many rows?
- Determine, how many numbers/characters/columns in a row?
- Determine, Increment/Decrement among the number of rows.
- Determine, starting in each row









Example: pattern

```
*
**
***
****
*****
```

Write a program to print given pattern

```
#include<stdio.h>
void main()
{
        int i,j;

        for(i=1;i<=5;i++)
        {
                for(j=1; j<=i; j++)
                {
                        printf("*");
                }

                printf("\n");
        }
}
```

Example: pattern

```
1
12
123
1234
12345
```

Write a program to print given pattern

```
#include<stdio.h>
void main()
{
        int i,j;

        for(i=1;i<=5;i++)
        {
                for(j=1; j<=i; j++)
                {
                        printf("%d",j);
                }

                printf("\n");
        }
}
```

Example: pattern

```
5
54
543
5432
54321
```

Write a program to print given pattern

```
#include<stdio.h>
void main()
{
        int i,j;

        for(i=5; i>0; i--)
        {
                for(j=5; j>=i; j--)
                {
                        printf("%d",j);
                }

                printf("\n");
        }
}
```

Example: pattern

```
    *
   **
  ***
 ****
*****
```

Write a program to print given pattern

```
#include<stdio.h>
void main()
{
        int i,j,k;

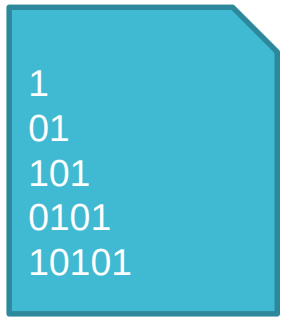
        for(i=1;i<=5;i++)
        {
                for(k=5;k>i;k--)
                {
                        printf(" ");
                }

                for(j=1;j<=i;j++)
                {
                        printf("*");
                }
                printf("\n")
        }
}
```

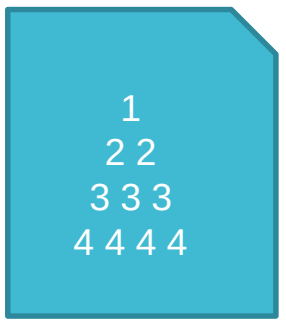
Example:

- Write a program to find sum of first N odd numbers. Ex. 1+3+5+7+……+N
- Write a program to find 1+1/2+1/3+1/4+….+1/n.
- Write a program to print all Armstrong numbers in a given range. For example 153 = 1^3 + 5^3 + 3^3.
- So, 153 is Armstrong number.
- Write a program to print given number in reverse order
- Write a program to check whether a given string is palindrome or not.
- Write a program to print Multiplication Table up to n.











Jump in Loops

- The number of times LOOP is repeated is decided in advance and the test condition is written to achieve this
- Sometimes, when executing a loop it becomes desirable to skip a part of the loop or to leave the loop as soon as certain condition occurs.
- C permits a jump from one statement to another statement within a loop as well as a jump out of loop
- Jumping out of loop: break / goto statement
- Skipping a part of loop: continue statement

Jumping out of loop

- The early exit from loop can be done using the break / the goto statement

break statement:

- When the break statement is encountered inside a loop, the loop is immediately terminated and the program continues with he statement immediately following the loop statement
- When the loops are nested, the break would only exit from the loop containing it, that is only single loop

goto statement:

- It can transfer the dcontrol to any place in a program
- It is useful to provide branching within a loop
- It is used to exit from deeply nested loops when error occurs, when break will not work



Jumping out of loop: break

```
while(condition)
{
        -------
        if(condition)
            break;
        ----
        -----
}
```

```
do
{
        -------
        if(condition)
            break;
        ----
        -----
}
while(condition);
```

```
for(----)
{
        -------
        if(error)
            break;
        ----
        -----
}
```

```
for(----)
{
        -------
        for(---)
        {
            if(---)
            break;
        ----
        }
        -----
}
```

Jumping out of loop: goto

```
while(condition)
{
        -------
        if(error)
            goto stop;

        if(condition)
            goto abc;
        ----
        -----
        abc:
        ------
        -----
}
stop:
----
```

```
for(----)
{
        -------
        for(---)
        {
            if(error)
                goto error;
        ----
        }
        -----
}

error:
-----
-----
```

Skipping a part of loop

- Continue statement:
- During the loop operations, it may be necessary to skip a part of a loop under certain conditions
- For example: Jop application- category. If person does not belong to particular category skip that part of the program
- For this, C has continue statement
- The continue, as the name implies, causes the loop to be continued with next iteration after skippking any statements in between



Skipping part of loop: continue

```
while(condition)
{
        -------
        if(condition)
            continue;
        ----
        -----
}
```

```
do
{
        -------
        if(condition)
            contonue;
        ----
        -----
}
while(condition);
```

```
for(----)
{
        -------
        if(-----)
            continue
        ----
        -----
}
```

goto statement

- goto is an virtual loop
- The goto statement allows us to transfer control of the program to the specified label.
- goto is keyword
- Syntax:

```
goto label;
.
.
.
label:
```

```
label:
.
.
.
goto label;
```

- The label is an identifier. When the goto statement is encountered, the control of the program jumps to label: and starts executing the code.

Example: goto

Write a program to print Odd numbers between 1 to n

```
#include<stdio.h>
void main()

{

        int  i,n;
        i=1;

        printf("Enter Number:");
        scanf("%d",&n);

        odd:
        if(i%2!=0)
        {
                printf("%d,",i);
        }
        i=i+1;

        if(i<=n)
        {
                goto odd;
        }
}
```

Example: goto

Write a program to find factors of a number

```
#include<stdio.h>
void main()
{
        int i,n;

        fact:
        if(n%i==0)
        {
                printf("%d,",i);
        }
        i=i+1;
        if(i<=n)
        {
                goto fact;
        }
}
```

o/p:
Enter number: 12
1, 2, 3, 4, 6, 12

#Thank You

From @:Darsha

☺☺☺